

Procédure de maintenance — Passerelle (onboardcc)

Document créé le 15/09/2026.

 **Valeurs masquées dans cette version publiée** (dépôt GitHub) : `<COMPTE>` = identifiant du compte d'hébergement o2switch, `<HOTE_O2SWITCH>` = nom d'hôte SSH du serveur, `<SERVEUR>` = son nom interne. À remplacer par les vraies valeurs avant de copier une commande.

Objectif de ce document

Ce document rassemble les gestes de maintenance **répétés régulièrement** une fois l'application **déjà installée et fonctionnelle** — contrairement à `1-procedure_deploiement_passerelle.md`, qui ne couvre que l'installation initiale ou une réinstallation complète depuis zéro.

Chaque section est **indépendante** : pas besoin de lire tout le document, seulement l'action concernée. C'est pensé pour un informaticien de la DCC qui n'a pas suivi le développement du projet — chaque geste explique ce qu'il fait et pourquoi, pas seulement la commande à taper.

 **Comprendre** : chaque encadré comme celui-ci apporte une explication de fond sur un concept technique utilisé dans le paragraphe qui précède.

 **RÈGLE À RESPECTER POUR TOUT MOT DE PASSE DE CE PROJET** : uniquement des lettres, des chiffres, et éventuellement `_ - .` — jamais d'autres caractères spéciaux (`# ? & @ : / % ! $` etc.). Voir §4 ci-dessous pour le détail de pourquoi cette règle existe.

Sommaire des actions

1. [Sauvegarde régulière](#)
2. [Déploiement d'un correctif ou d'une évolution fournie sur GitHub](#)
 - 2.1 à 2.7 : gestes de base (connexion, `git pull`, dépendances, build backend/frontend, sanity checks)
 - 2.8 [Si une migration de base de données est nécessaire](#)
 - 2.9 [Ajouter ou modifier une variable d'environnement](#)
3. [Contrôles à faire en cas de remontée utilisateur](#)
 - 3.1 [Vérifier que le serveur tourne](#)
 - 3.2 [Si l'application ne répond plus](#)
 - 3.3 [Vérifier le certificat SSL](#)
4. [Sécurité](#)
 - 4.1 [Règle des mots de passe de ce projet](#)
 - 4.2 [Changer le mot de passe de la base de données](#)
 - 4.3 [Réinitialiser le mot de passe PostgreSQL local \(poste de développement uniquement\)](#)
 - 4.4 [Certificat SSL](#)
 - 4.5 [Précaution encodage — éviter la corruption des caractères accentués en base](#)
5. [Journal des livraisons en VAL et en production](#)

1. Sauvegarde régulière

⚠ Section à compléter. Pas encore rédigée — objectif : décrire la fréquence, le mécanisme (dump PostgreSQL + dossier `uploads/`) et l'endroit de stockage des sauvegardes régulières de l'application en production.

2. Déploiement d'un correctif ou d'une évolution fournie sur GitHub

Cas d'usage : un correctif ou une nouvelle fonctionnalité a été développé (par Damien, ou plus tard par un autre développeur) et poussé (`push`) sur le dépôt GitHub

`github.com/DamienMoreauConsultant/onboardcc`, branche `main`. Cette procédure le fait passer sur le serveur `o2switch` (VAL) puis, une fois validé, sur le serveur de production DCC — même méthode pour les deux, seule l'adresse change.

💡 Rappel du principe : un seul sens d'écriture vers GitHub (le poste de développement). Les serveurs (`o2switch`, DCC) ne font **jamais** que `pull` — ils n'écrivent jamais de code directement.

2.1 Se connecter et se placer dans le bon dossier

```
ssh -i C:\Users\DamienRockstar\.ssh\passerelle_pc_damien <COMPTE>@<HOTE_O2SWITCH>
```

💡 Demande la phrase secrète de la clé. Résultat attendu : l'invite affiche `[<COMPTE>@<SERVEUR> ~]$`.

```
source /home/<COMPTE>/nodeenv/passerelle-api/24/bin/activate && cd ~/passerelle-api
```

💡 `source` active l'environnement Node isolé (version 24) pour que `node/npm/pnpm` utilisent la bonne version dans cette session. Résultat attendu : `(24)` apparaît au début de l'invite, et tu es maintenant **dans le dossier du dépôt Git** (`~/passerelle-api`) — c'est important, voir l'encadré ci-dessous.

💡 **Pourquoi se placer dans ce dossier précis avant tout `git pull`.** `git pull` doit être lancé **depuis l'intérieur d'un dépôt Git** (un dossier contenant un sous-dossier caché `.git`). Le code a été cloné une seule fois, lors de l'installation initiale, dans `~/passerelle-api` (voir 1-procedure_deploiement_passerelle.md §2) — nulle part ailleurs sur le serveur. Lancer `git pull` depuis `~` (le dossier personnel, juste après la connexion SSH) échoue avec `fatal: not a git repository` : Git remonte les dossiers parents à la recherche d'un `.git`, n'en trouve pas, et abandonne à la limite du disque (`/home`).

2.2 Vérifier l'état avant de récupérer le nouveau code

```
git status
```

Résultat attendu : `nothing to commit, working tree clean`. Le serveur ne doit **jamais** avoir de modification locale non committée (rappel : il n'écrit jamais de code lui-même) — si ce n'est pas clean,

s'arrêter et comprendre pourquoi avant de continuer (ne pas écraser une modification qui aurait été faite par erreur directement sur le serveur).

2.3 Récupérer le nouveau code

⚠ **Avant le `git pull` : vérifier que le dépôt GitHub est bien à jour** de la version à déployer (dernier commit visible sur la page du dépôt) — le serveur ne récupère que ce qui a été poussé.

```
git pull
```

Résultat attendu : la liste des fichiers modifiés/crétés s'affiche (ex. `6 files changed, 64 insertions(+), 30 deletions(-)`). Si le message est `Already up to date.`, c'est qu'il n'y avait rien de nouveau à récupérer — vérifier que le `push` a bien été fait côté GitHub avant de s'inquiéter.

⚠ **Si `git pull` échoue avec `error: Your local changes to the following files would be overwritten by merge`** : malgré la vérification §2.2, le serveur a bien une modification locale non committée sur le(s) fichier(s) cité(s). **Ne jamais faire `git add`/`git commit` sur le serveur** (ça violerait le principe du serveur "lecture seule" côté Git, voir §2 ci-dessus). À la place :

1. Regarder ce qui diffère : `git diff <fichier>`.
2. Comparer avec le changement qui arrive (déjà visible dans le `git pull` refusé, ou `git log -p origin/main -1 -- <fichier>`).
3. **Si le changement local est déjà inclus dans ce qui arrive** (cas vécu le 15/09/2026 sur `pnpm-workspace.yaml`, généré par `pnpm approve-builds` — voir encadré §2.4 ci-dessous) : `git restore <fichier>` (annule la modification locale, sans risque), puis relancer `git pull`.
4. **Si le changement local est différent et doit être conservé** : ne pas trancher seul — c'est le signe qu'une correction a été faite directement sur le serveur sans passer par GitHub, ce qui ne devrait jamais arriver. Reporter ce changement à la source (le committer depuis un PC de développement, le pousser, puis repuller ici) plutôt que de le garder localement.

2.4 Si les dépendances ont changé (`package.json`, `pnpm-lock.yaml`, `pnpm-workspace.yaml`)

💡 **Cas vécu le 15/09/2026** : `pnpm approve-builds` (voir 1-procedure_deploiement_passerelle.md §2) modifie `pnpm-workspace.yaml` **localement sur le serveur**. Lors de l'installation initiale, ce changement n'avait pas été rapatrié vers GitHub — il est resté comme modification locale orpheline sur `o2switch`, jusqu'à entrer en conflit avec un `git pull` ultérieur (voir l'encadré juste au-dessus pour la procédure de résolution). Depuis, le réflexe "vérifier `git status` après tout `pnpm approve-builds`" a été ajouté à la procédure d'installation — ça ne devrait plus se reproduire sur un nouvel environnement, mais bon à savoir si un cas similaire apparaît sur un autre fichier.

Seulement si `git pull` a modifié un de ces 3 fichiers (sinon, passer directement à 2.5/2.6) :

⚠ **Si tu n'es pas déjà connecté à l'environnement Node.js de Passerelle dans cette fenêtre** (repère : l'invite ne commence pas par (24)), exécute d'abord la commande `source` du §2.1 avant

`pnpm install` — sinon `-bash: pnpm: command not found`. L'activation ne dure que le temps de la session SSH en cours, il faut la refaire à chaque nouvelle connexion.

```
pnpm install
```

💡 Réinstalle uniquement ce qui a changé — pas besoin de tout retélécharger. Si le message `ERR_PNPM_UNAUTHORIZED_ENV` ou un script bloqué apparaît, voir 1-procedure_deploiement_passerelle.md §2 (`pnpm approve-builds`).

2.5 Si le backend a changé (`artifacts/api-server/...`)

```
cd artifacts/api-server
pnpm run build
cd ~/passerelle-api
```

💡 Régénère `dist/index.mjs`, le fichier réellement exécuté par le serveur — le code source (`src/`) seul ne suffit pas, il doit être recompilé.

Puis **redémarrer l'application** : cPanel → **Setup Node.js App** → l'application concernée → bouton **"REDÉMARRER"**. Sans ce redémarrage, l'ancien code continue de tourner même après le build.

2.6 Si le frontend a changé (`artifacts/onboardcc/...`)

```
cd ~/passerelle-api/artifacts/onboardcc
export PORT=5000
export BASE_PATH=/
export VITE_API_BASE_URL=/api
pnpm run build
```

💡 `BASE_PATH` et `VITE_API_BASE_URL` doivent être identiques à l'installation initiale (voir 1-procedure_deploiement_passerelle.md §7.1) — une valeur différente casserait le chargement de l'interface.

```
cp -r dist/public/* /home/<COMPTE>/passerelle.dm-informatique.fr/
```

💡 Recopie les fichiers construits vers le dossier réellement servi par Apache. **Ce dossier n'est jamais lu directement depuis le dépôt Git** — sans cette copie, `git pull` seul ne change rien à ce que voient les visiteurs, même après un build réussi.

Pas de redémarrage nécessaire pour le frontend (fichiers statiques, servis directement par Apache).

2.7 Sanity checks — vérifier que ça fonctionne réellement

À dérouler après **tout** déploiement, avant de considérer l'action terminée. Sur des données **TEST_*** uniquement, avec nettoyage après coup si des données ont été créées.

#	Test	Comment	Attendu
1	Le serveur répond	<code>curl -s https://passerelle.dm-informatique.fr/api/healthz</code>	<code>{"status":"ok"}</code>
2	L'interface se charge	Ouvrir <code>https://passerelle.dm-informatique.fr/</code> dans un navigateur	La page de connexion s'affiche, sans erreur dans la console (F12)
3	Connexion fonctionne	Se connecter avec un compte de test existant	Accès à l'écran attendu selon le rôle
4	La zone modifiée fonctionne	Selon la nature du correctif — ex. écran visuellement corrigé, nouveau bouton actif...	Comportement conforme à ce qui était demandé
5	Rien d'autre n'est cassé	Reparcourir rapidement 2-3 écrans clés sans rapport avec le correctif	Aucune régression visible

💡 Pour une recette plus complète (parcours métier bout en bout : import, matching, affectation), voir la checklist détaillée dans `1-procedure_deploiement_passerelle.md` §8.3 — à réutiliser après une évolution majeure, pas nécessaire après un petit correctif visuel.

2.8 Si une migration de base de données est nécessaire

Cas d'usage : le correctif ajoute ou modifie une colonne/contrainte (fichier `artifacts/api-server/migrations/0XX_...sql` dans le code livré). Toujours **après** le `git pull` (§2.3) et **avant** de redémarrer le backend (§2.5) — le nouveau code peut dépendre du nouveau schéma.

1. Transférer le fichier de migration vers le serveur, depuis un PC local (PowerShell, pas la session SSH) :

```
scp -i C:\Users\DamienRockstar\.ssh\passerelle_pc_damien
"chemin\vers\migrations\0XX_nom.sql" <COMPTE>@<HOTE_O2SWITCH>:~/
```

2. Se connecter à la base, depuis la session SSH déjà ouverte (§2.1) :

```
psql -h localhost -U <COMPTE>_passerelle_app -d <COMPTE>_passerelle
```

3. Charger le fichier :

```
\i 0XX_nom.sql
```

💡 Chaque migration de ce projet contient elle-même ses garde-fous (aperçu, nettoyage préalable si besoin) — lire les commentaires en tête du fichier avant de lancer, ils expliquent ce que fait la migration et pourquoi. Si le fichier ne se termine pas par un **COMMIT** ; explicite, le taper soi-même après avoir lu le résultat, comme pour tout script de ce projet.

4. Vérifier : `pnpm run check-seed` (depuis `artifacts/api-server`, voir §2.1 pour l'activation de l'environnement Node) doit confirmer le bon nombre de tables.

⚠️ Si la migration (ou tout autre besoin ponctuel) t'amène à taper une requête SQL contenant du texte accentué directement dans l'invite `psql`, voir d'abord §4.5 — risque de corruption silencieuse des caractères sur Windows.

💡 **Ce n'est jamais un paquet npm/pnpm à installer** : une migration ne modifie que la base de données (schéma ou données), jamais le code. Aucune dépendance particulière n'est à installer sur le serveur pour qu'une fonctionnalité déjà présente dans le code (par exemple les info-bulles d'aide de la fiche de vœux, `Retour_23`) s'affiche correctement — si une bibliothèque front est en cause, elle est déjà déclarée dans `package.json` et récupérée automatiquement par le `pnpm install` du §2.4, comme n'importe quelle autre dépendance. Exemple concret : la migration `018_aide_contextuelle_voeux_texte.sql` (23/09/2026) ne fait qu'upsert le texte réel des 23 tooltips de la fiche de vœux dans la table `aide_contextuelle` — elle doit être rejouée avec la procédure ci-dessus sur chaque base existante (locale et o2switch) pour que ces tooltips apparaissent, exactement comme pour n'importe quelle autre migration.

2.9 Ajouter ou modifier une variable d'environnement (ex. nouveau barème de score)

Cas d'usage : le correctif introduit une nouvelle variable `MATCH_SCORE_*` (ou en modifie une) — voir `artifacts/api-server/.env.example` dans le code livré pour la liste à jour et les valeurs attendues.

1. cPanel → **Setup Node.js App** → l'application `passerelle.dm-informatique.fr` → **Environnement variables**.

2. Pour une **nouvelle** variable : bouton **"Ajouter une variable"**, renseigner le nom exact (ex. `MATCH_SCORE_DURATION_SAME_LEVEL`) et la valeur (voir `.env.example`) → **Enregistrer**. Pour **modifier** une variable existante : icône crayon sur la ligne concernée.

3. L'application redémarre automatiquement à l'enregistrement des variables — pas de bouton "Redémarrer" séparé à cliquer pour ce cas précis (contrairement à un déploiement de code, §2.5).

⚠️ **Ne jamais oublier une variable** : `scoreConfig()` (`matching.ts`) vérifie **toutes** les variables `MATCH_SCORE_*` d'un coup au moment du calcul — il suffit qu'une seule manque ou soit invalide pour que **tout** recalcul échoue (pas seulement le critère concerné), avec l'erreur `Configuration de scoring absente ou invalide : NOM_DE_LA_VARIABLE`. Comparer soigneusement `.env.example` à ce qui est déjà dans cPanel avant de considérer l'étape terminée.

4. Vérifier : recalculer une opportunité de test et confirmer que le nouveau barème s'applique (voir §2.7).

3. Contrôles à faire en cas de remontée utilisateur

Un utilisateur signale que "ça ne marche pas" — voici l'ordre des vérifications à faire, de la plus rapide à la plus poussée.

3.1 Vérifier que le serveur tourne

1. Se connecter en SSH et activer l'environnement (voir §2.1 ci-dessus).
2. Tester que l'API répond :

```
curl -s https://passerelle.dm-informatique.fr/api/healthz
```

💡 `curl` envoie une requête web depuis le terminal, comme le ferait un navigateur, et affiche la réponse brute — plus rapide qu'ouvrir un navigateur, et sans le souci de certificat SSL rencontré dans Chrome (voir §3.3 si le souci est justement le certificat). Résultat attendu : `{"status":"ok"}`.

3.2 Si l'application ne répond plus

Dans cPanel → **Setup Node.js App** → l'application `passerelle.dm-informatique.fr` : bouton **"REDÉMARRER"**.

Si ça ne suffit pas, consulter cPanel → **"Erreurs"** (barre de recherche), puis si besoin reproduire le lancement manuellement en SSH pour voir le vrai message d'erreur Node.js (invisible autrement sur cet hébergement mutualisé) :

```
cd ~/passerelle-api
export PORT=3000
export DATABASE_URL='...' # guillemets SIMPLES obligatoires
# ... les autres variables d'environnement (voir cPanel → Environment variables
pour la liste) ...
node artifacts/api-server/dist/index.mjs
```

💡 Cette méthode a permis de diagnostiquer un vrai bug (variable `PORT` manquante) alors que cPanel n'affichait qu'un message générique et inexploitable — voir [1-procedure_deploiement_passerelle.md §7](#) pour le détail de ce cas réel.

3.3 Vérifier le certificat SSL

Symptôme : le navigateur affiche *"Votre connexion n'est pas privée"* / `NET::ERR_CERT_AUTHORITY_INVALID`, ou `curl` échoue avec `SSL certificate problem: self signed certificate`.

Vérifier : cPanel → recherche **"SSL/TLS Status"** → repérer la ligne du domaine concerné. Si elle affiche **"Self-signed"** avec la mention *"will not renew via AutoSSL because it was not issued via AutoSSL"*, c'est la cause.

💡 **Comprendre** : un nouveau sous-domaine reçoit d'abord un certificat auto-signé temporaire. **AutoSSL** (le système qui délivre automatiquement de vrais certificats Let's Encrypt) **ignore volontairement** tout domaine qui a déjà un certificat non-AutoSSL — même auto-signé — pour ne

pas remplacer un certificat que quelqu'un aurait installé exprès. Résultat : sans intervention, ça ne se corrige jamais tout seul, même après plusieurs jours.

Voir §4.4 ci-dessous pour la procédure de correction (c'est un geste de sécurité, classé avec les autres actions de cette catégorie).

4. Sécurité

4.1 Règle des mots de passe de ce projet

Uniquement des lettres, des chiffres, et éventuellement `_ - .` — jamais d'autres caractères spéciaux (`# ? & @ : / % ! $ ' etc.`), pour **tout** mot de passe créé pour ce projet (base de données, comptes email, etc.).

Pourquoi une règle aussi stricte : un mot de passe généré avec des caractères "forts" classiques a provoqué deux bugs distincts lors du déploiement initial — `!` interprété par le terminal (bash) comme un rappel d'historique de commande, puis `#/?/&` cassant la lecture de `DATABASE_URL` comme une URL (`#`=début de fragment, `?`=début de paramètres, `&`=séparateur de paramètres — tous ont un sens réservé dans une URL). Résultat : connexion impossible, message d'erreur générique `Invalid URL` très difficile à relier à la vraie cause sans un diagnostic poussé.

Les seuls caractères non alphanumériques **sûrs partout** (terminal, URL, SQL) sont `_ - .` — s'en tenir à ceux-là évite ce risque entièrement, sans sacrifier la sécurité (la longueur du mot de passe compte bien plus que la diversité des symboles utilisés).

💡 Depuis le 23/09/2026 (Retour_25), cette même règle est appliquée en code aux mots de passe des comptes applicatifs (candidat, recruteur, CM, admin) — plus seulement conseillée pour les mots de passe d'infrastructure choisis à la main. Exigée à l'inscription/réinitialisation : 12 caractères minimum, avec au moins une minuscule, une majuscule, un chiffre et un des 3 caractères `_ - .` — aucun autre caractère spécial n'est accepté (`auth.ts`, constante `PASSWORD_RULE`).

4.2 Changer le mot de passe de la base de données

1. Choisir le nouveau mot de passe, en respectant la règle §4.1.

2. L'appliquer côté PostgreSQL, connecté à `psql` :

```
ALTER USER <COMPTE>_passerelle_app WITH PASSWORD 'NOUVEAU_MOT_DE_PASSE';
```

💡 Résultat attendu : `ALTER ROLE` affiché en retour.

3. Mettre à jour la variable dans cPanel — Setup Node.js App → l'application → Environment variables → modifier `DATABASE_URL` (icône crayon) → remplacer uniquement le mot de passe dans :

```
postgres://<COMPTE>_passerelle_app:NOUVEAU_MOT_DE_PASSE@localhost:5432/<COMPTE>_passerelle
```

→ **ENREGISTRER.**

4. Vérifier : retester la connexion réelle sur <https://passerelle.dm-informatique.fr/> (l'application redémarre automatiquement à l'enregistrement des variables).

4.3 Réinitialiser le mot de passe PostgreSQL local (poste de développement uniquement)

⚠ Ne concerne pas le serveur o2switch/DCC — uniquement un PC de développement qui a sa propre installation locale de PostgreSQL. Utile si le mot de passe du compte **postgres** n'a pas été correctement défini/retenu à l'installation (fréquent avec une installation "silencieuse" via **winget**, sans l'assistant graphique habituel). Conservé ici pour mémoire au cas où un futur développeur en aurait besoin.

1. PowerShell en administrateur (clic droit → "Exécuter en tant qu'administrateur") — nécessaire pour modifier un fichier système et redémarrer un service.

2. Autoriser temporairement une connexion sans mot de passe :

```
notepad "C:\Program Files\PostgreSQL\16\data\pg_hba.conf"
```

Remplacer **scram-sha-256** par **trust** sur les deux lignes suivantes uniquement (ne rien changer d'autre) :

host	all	all	127.0.0.1/32	scram-sha-256
host	all	all	:::1/128	scram-sha-256

Enregistrer, fermer, puis appliquer :

```
Restart-Service postgresql-x64-16
```

💡 trust autorise n'importe quel programme du PC à se connecter sans mot de passe — mode temporaire uniquement, à ne jamais laisser en place.

3. Se connecter (sans mot de passe cette fois) et définir le nouveau :

```
& "C:\Program Files\PostgreSQL\16\bin\psql.exe" -U postgres
```

```
ALTER USER postgres WITH PASSWORD 'NOUVEAU_MOT_DE_PASSE';
```

(règle de caractères habituelle, §4.1 — jamais le même mot de passe qu'un autre environnement)

4. Remettre la sécurité en place — rouvrir le même fichier, remettre **scram-sha-256** à la place de **trust** sur les deux lignes modifiées, enregistrer, puis :

```
Restart-Service postgresql-x64-16
```

5. Vérifier : relancer `psql -U postgres` — le mot de passe doit maintenant être demandé et fonctionner avec la nouvelle valeur.

4.4 Certificat SSL

Symptôme et diagnostic : voir §3.3 ci-dessus.

Corriger (procédure testée et fonctionnelle) : cPanel → recherche **"Let's Encrypt"** → outil **"Let's Encrypt™ SSL"** → section **"Générer un nouveau certificat"** → rechercher le nom de domaine → cliquer **"Générer"** → garder les options par défaut (les deux domaines proposés cochés, méthode HTTP-01, clé RSA 2048-bit) → **"Générer"** (pas "Simuler la génération"). Apache redémarre automatiquement en arrière-plan après installation — attendre quelques secondes avant de retester.

⚠ Ce piège concerne potentiellement **tout nouveau domaine créé sur ce type d'hébergement** — à garder en tête pour le futur serveur DCC s'il utilise une infrastructure cPanel similaire.

4.5 Précaution encodage — éviter la corruption des caractères accentués en base

⚠ **Concerne tout le monde qui se connecte à psql** (poste de développement local **ou** session SSH sur o2switch/DCC) **dès qu'un caractère accentué (é, è, à, ç...) est en jeu** — à lire avant toute intervention manuelle en base, en particulier par un futur informaticien DCC qui découvrirait cette procédure sans connaître l'historique du projet.

Le risque : sur Windows, `psql` peut mal interpréter les caractères accentués si l'encodage de la console ne correspond pas à celui utilisé par la base (UTF-8 partout dans ce projet). Le symptôme le plus visible est l'avertissement affiché à la connexion :

```
Attention : l'encodage console (xxx) diffère de l'encodage Windows (xxx).
```

Mais le vrai danger est **silencieux** : sans cet avertissement (ex. en connexion SSH vers le Linux d'o2switch, où `psql` ne l'affiche pas), un caractère accentué **tapé au clavier** dans une requête peut malgré tout être mal encodé avant même de partir vers le serveur, et venir corrompre une donnée en base sans aucun message d'erreur.

Règle 1 (la vraie protection) — ne jamais taper une requête contenant du texte accentué directement dans l'invite psql. Toujours préparer un fichier `.sql` à l'avance (comme le pattern déjà utilisé pour les migrations, §2.8) et le charger avec :

```
\i chemin/vers/le_fichier.sql
```

Un fichier échappe entièrement au risque de frappe clavier — vérifier seulement que l'éditeur de texte utilisé pour le créer l'a bien enregistré en **UTF-8** (le Bloc-notes Windows propose parfois "ANSI" par défaut, ce qui

recréerait le même problème en sens inverse).

Règle 2 (filet de sécurité) — si une requête manuelle ponctuelle est vraiment nécessaire (ex. un `SELECT` de diagnostic), forcer l'encodage avant de se connecter :

```
$env:PGCLIENTENCODING = "UTF8"
```

💡 Ne dure que le temps de la fenêtre PowerShell ouverte — à refaire à chaque nouvelle session. Corrige l'encodage utilisé pour échanger avec la base (`client_encoding`), mais ne change pas l'encodage de la console elle-même — pour ça, `chcp 65001` dans la même fenêtre, **avant** toute connexion SSH ou tout lancement de `psql`, si le message d'avertissement persiste malgré cette variable.

5. Journal des livraisons en VAL et en production

Objectif : garder la trace de **ce qui a réellement été déployé, quand, et par quelle action précise** — contrairement aux sections 1 à 4 ci-dessus qui décrivent des gestes **génériques** et réutilisables, ce journal enregistre **chaque livraison concrète**. En cas de souci après une mise à jour, ce journal permet de savoir immédiatement ce qui a changé sans devoir comparer l'historique Git à la main.

💡 **Damien (19/09/2026)** : suivi formel commencé à la première mise en production réelle — pas nécessaire de renseigner ce journal pour les allers-retours de test sur VAL avant cette date. Le support (structure ci-dessous) est prêt dès maintenant pour ne pas avoir à l'improviser le jour venu.

Modèle à copier pour chaque nouvelle livraison (un sous-paragraphe par livraison, dans l'ordre chronologique — ne jamais réécrire une entrée déjà renseignée, uniquement en ajouter de nouvelles) :

```
### JJ/MM/AAAA – <VAL ou Production> – <objet court, ex. "correctifs suite retours DCC #01-13">
```

```
**Objet** : <une phrase – correctifs suite à remontée DCC / évolution fonctionnelle / élévation de version d'un composant (Node, PostgreSQL...)>
```

```
**Contenu** :
```

```
- <liste des fichiers/dossiers concernés ou des Retours de suivi couverts, ex. "Retour_01 à Retour_13 (voir 20260918_suivi_des_tests.md)">
```

```
**Procédure appliquée** (cocher ce qui s'applique, avec renvoi vers le geste générique) :
```

```
- [ ] Code : §2.1-2.3 (git pull) + §2.5 (backend) et/ou §2.6 (frontend)
- [ ] Dépendances : §2.4 (pnpm install)
- [ ] Migration(s) SQL : §2.8 – fichier(s) : <nom(s)>
- [ ] Variable(s) d'environnement : §2.9 – variable(s) : <nom(s)>
- [ ] Sanity checks : §2.7 réalisés, OK
```

****Résultat**** : <OK / problème rencontré et lien vers sa résolution>

(aucune livraison enregistrée à ce jour — première entrée à créer lors de la mise en production)

Journal des mises à jour de ce document

Date	Modification
15/09/2026	Création du document, en scindant l'ancienne "Annexe A — Maintenance courante (FAQ)" de <code>1-procedure_deploiement_passerelle.md</code> . Ajout de la section 2 "Déploiement d'un correctif" (absente jusqu'ici — seule la procédure de <code>git clone</code> initiale existait, pas celle de mise à jour via <code>git pull</code>). Section 1 "Sauvegarde régulière" créée en stub, à compléter.
15/09/2026	Section 2 validée en conditions réelles sur o2switch (premier déploiement d'un correctif frontend — couleurs et logo DCC) : conflit <code>git pull/pnpm-workspace.yaml</code> rencontré et documenté (§2.3), rappel d'activation de l'environnement Node ajouté (§2.4). Déploiement conclu avec succès (<code>{"status": "ok"}</code> , interface vérifiée visuellement).
19/09/2026	Ajout (demande de Damien) : §2.8 "Si une migration de base de données est nécessaire" et §2.9 "Ajouter ou modifier une variable d'environnement" — gestes génériques manquants, nécessaires pour déployer le lot de correctifs Retour_01 à Retour_13 (migrations 015/016, nouvelles variables <code>MATCH_SCORE_*</code>). Ajout de la section 5 "Journal des livraisons en VAL et en production" (modèle vierge, à commencer formellement à la première mise en production réelle, sur décision de Damien — mais support préparé dès maintenant).
19/09/2026	Ajout §4.5 "Précaution encodage" (demande de Damien, après avoir rencontré l'avertissement <code>psql</code> d'encodage console en local) : anticipe le même risque pour un futur informaticien DCC qui interviendrait manuellement en base de production. Règle principale : ne jamais taper de texte accentué directement dans l'invite <code>psql</code> , toujours passer par un fichier <code>.sql</code> (même pattern que §2.8) ; <code>\$env:PGCLIENTENCODING/chcp 65001</code> en filet de sécurité si une requête manuelle ponctuelle est inévitable. Référencée depuis §2.8.
23/09/2026	Ajout d'une note dans §2.8 (retour Damien, Retour_23 — hypothèse d'un composant à installer pour que les tooltips s'affichent) : rappel qu'une migration ne modifie jamais le code ni ses dépendances, seulement la base ; toute bibliothèque front nécessaire est déjà déclarée dans <code>package.json</code> et récupérée par le <code>pnpm install</code> du §2.4. Illustré avec la migration <code>018_aide_contextuelle_voeux_texte.sql</code> , à rejouer sur chaque base existante pour faire apparaître le texte réel des tooltips de la fiche de vœux.
23/09/2026	Ajout d'une note dans §4.1 (Retour_25) : la règle de caractères déjà en vigueur pour les mots de passe d'infrastructure (lettres, chiffres, <code>_</code> - <code>.</code> uniquement) est désormais aussi appliquée en code aux comptes applicatifs, avec en plus une longueur minimale de 12 caractères et une exigence de complexité (minuscule + majuscule + chiffre + spécial).
25/09/2026	Simplification du rappel de principe (§2) et ajout, en tête du §2.3, du rappel de vérifier que GitHub est à jour avant <code>git pull</code> .